TP1 : Projet fédérateur ML2

Introduction au Deep Learning et exploration des données MNIST

Matière : Machine Learning 2 (Deep Learning)
Bibliothèques : NumPy, Matplotlib, TensorFlow/Keras

1. Présentation du projet fédérateur

1.1. Contexte général

L'apprentissage automatique (Machine Learning) et l'apprentissage profond (Deep Learning) occupent aujourd'hui une place importante dans le développement des systèmes intelligents. Ces technologies permettent aux ordinateurs d'apprendre à partir de données afin de réaliser des tâches telles que la reconnaissance d'images, la détection d'anomalies, la traduction automatique ou encore la prédiction de phénomènes complexes.

Par exemple, un système de reconnaissance de chiffres manuscrits doit être capable d'identifier un chiffre même lorsque son écriture varie d'une personne à une autre.

Comment un ordinateur peut-il reconnaître automatiquement un chiffre manuscrit alors qu'il ne possède pas les capacités de perception d'un être humain ?

Pour résoudre ce problème, nous allons utiliser des techniques de Deep Learning permettant d'apprendre automatiquement les caractéristiques des images.

1.2. Objectif du projet

Le projet fédérateur ML2 consiste à concevoir et développer progressivement un système intelligent de reconnaissance de chiffres manuscrits.

Au cours du semestre, les étudiants réaliseront les différentes étapes d'un projet de Deep Learning :

1. Exploration et compréhension des données.
2. Prétraitement et normalisation.
3. Construction d'un premier réseau de neurones.
4. Entraînement et optimisation.
5. Évaluation des performances.
6. Construction d'un réseau de neurones convolutif (CNN).
7. Comparaison et amélioration des modèles.

8. Présentation du projet final.

Chaque séance permettra d'enrichir progressivement le même projet.

2. Objectifs pédagogiques du TP1

À l'issue de cette séance, l'étudiant sera capable de :

- Comprendre les principes fondamentaux du Machine Learning et du Deep Learning.
- Distinguer un problème de classification d'un problème de régression.
- Comprendre la notion d'apprentissage supervisé.
- Expliquer comment une image numérique est représentée.
- Charger et explorer un jeu de données.
- Visualiser des images avec Python.
- Analyser la distribution des classes d'un jeu de données.

3. Rappel-cours : notions fondamentales

3.1. Machine Learning et Deep Learning

Le Machine Learning est une branche de l'intelligence artificielle qui permet à un système informatique d'apprendre à partir d'exemples plutôt que de suivre uniquement des règles programmées explicitement.

Prenons l'exemple d'un système chargé de reconnaître les chiffres manuscrits.

Dans une approche traditionnelle, il faudrait définir manuellement des règles permettant de reconnaître chaque chiffre. Cependant, cette approche devient difficile lorsque les chiffres présentent des formes et des écritures différentes.

Avec le Machine Learning, nous fournissons au programme un ensemble d'images accompagnées de leurs étiquettes. Le programme apprend alors à établir une relation entre les images et les chiffres correspondants.

Le Deep Learning est une sous-catégorie du Machine Learning qui repose sur l'utilisation de réseaux de neurones comportant plusieurs couches de traitement.

Ces réseaux sont capables d'apprendre progressivement des représentations complexes des données.

Exemple concret :

Dans un système de reconnaissance d'images, les premières couches d'un réseau peuvent apprendre à détecter des caractéristiques simples, comme des lignes ou des

contours. Les couches suivantes peuvent combiner ces caractéristiques pour identifier des formes plus complexes.

3.2. Apprentissage supervisé

L'apprentissage supervisé consiste à entraîner un modèle à partir de données dont les réponses correctes sont connues.

Chaque exemple comprend :

- Une entrée : les données fournies au modèle.
- Une étiquette : la réponse associée à ces données.

Dans notre projet :

- L'entrée est une image représentant un chiffre manuscrit.
- L'étiquette est le chiffre correspondant, compris entre 0 et 9.

L'objectif est de construire un modèle capable de prédire correctement l'étiquette d'une nouvelle image.

3.3. Classification et régression

En Machine Learning, deux grandes catégories de problèmes supervisés sont la classification et la régression.

Classification : le modèle doit prédire une catégorie.

Exemples :

- Reconnaître un chiffre manuscrit.
- Identifier un courrier électronique comme spam ou non-spam.
- Classer une image comme représentant un chat ou un chien.

Régression : le modèle doit prédire une valeur numérique continue.

Exemples :

- Prédire le prix d'une maison.
- Estimer la température du lendemain.
- Prévoir la consommation électrique d'un bâtiment.

Notre projet est un problème de classification multiclasse, car nous cherchons à distinguer dix catégories différentes.

3.4. Représentation numérique d'une image

Une image numérique est constituée de pixels.

Dans une image en niveaux de gris, chaque pixel possède généralement une intensité comprise entre 0 et 255 :

- 0 représente le noir.
- 255 représente le blanc.
- Les valeurs intermédiaires représentent différents niveaux de gris.

Une image de dimensions 28 × 28 contient 784 pixels.

Elle peut être représentée sous la forme d'une matrice de 28 lignes et 28 colonnes.

Cette représentation numérique permet aux algorithmes de Machine Learning de manipuler les images et d'apprendre à reconnaître leurs caractéristiques.

3.5. Données d'entraînement et données de test

Lors du développement d'un modèle de Machine Learning, les données sont généralement séparées en plusieurs ensembles.

Ensemble d'entraînement (Training Set)

Il contient les exemples utilisés pour apprendre les paramètres du modèle.

Ensemble de validation (Validation Set)

Il permet de suivre les performances pendant le développement et d'aider à sélectionner les paramètres et les configurations du modèle.

Ensemble de test (Test Set)

Il permet d'évaluer les performances finales du modèle sur des données qui n'ont pas été utilisées pour son développement.

Cette séparation est importante pour vérifier que le modèle est capable de généraliser ses connaissances à de nouvelles données.

4. Partie pratique : découverte du jeu de données MNIST

4.1. Présentation du jeu de données

MNIST est un jeu de données de référence pour l'apprentissage de la classification d'images.

Il contient 70 000 images de chiffres manuscrits :

- 60 000 images pour l'entraînement.
- 10 000 images pour le test.

Chaque image possède une résolution de 28 × 28 pixels et appartient à l'une des dix classes correspondant aux chiffres de 0 à 9.

Étape 1 — Préparation de l'environnement

Ouvrir Google Colab à l'adresse :

https://colab.research.google.com/

Créer un nouveau notebook nommé :

Projet_Federateur_ML2_TP1.ipynb

Importer les bibliothèques nécessaires :

```
import numpy as np

import matplotlib.pyplot as plt

import tensorflow as tf


from tensorflow.keras.datasets import mnist


print("Version TensorFlow :", tf.__version__)
```

Questions :

1. Quel est le rôle de NumPy ?
2. À quoi sert Matplotlib ?
3. Pourquoi utilisons-nous TensorFlow dans ce projet ?

Étape 2 — Chargement des données

Charger le jeu de données MNIST :

```
(x_train, y_train), (x_test, y_test) = mnist.load_data()


print("Données chargées")
```

Les variables obtenues sont :

- x_train : images d'entraînement.
- y_train : étiquettes des images d'entraînement.
- x_test : images de test.

- y_test : étiquettes des images de test.

Travail demandé :

Expliquer le rôle de chacune de ces variables.

Étape 3 — Exploration des dimensions

Exécuter le programme suivant :

```
print("Dimensions x_train :", x_train.shape)

print("Dimensions y_train :", y_train.shape)


print("Dimensions x_test :", x_test.shape)

print("Dimensions y_test :", y_test.shape)
```

Questions :

1. Combien d'images sont disponibles pour l'entraînement ?
2. Combien d'images sont disponibles pour le test ?
3. Que représente la dimension (60000, 28, 28) ?
4. Pourquoi les étiquettes possèdent-elles une seule dimension ?

Étape 4 — Visualisation d'une image

Afficher la première image du jeu de données :

```
plt.imshow(x_train[0], cmap="gray")


plt.title("Chiffre : " + str(y_train[0]))


plt.axis("off")


plt.show()
```

Travail demandé :

1. Identifier le chiffre représenté.
2. Modifier le programme pour afficher l'image d'indice 25.
3. Afficher l'étiquette associée à cette image.

Étape 5 — Visualisation de plusieurs images

Exécuter le programme suivant :

```
plt.figure(figsize=(10, 4))


for i in range(10):
  plt.subplot(2, 5, i + 1)
  plt.imshow(x_train[i], cmap="gray")
  plt.title(f"Chiffre : {y_train[i]}")
  plt.axis("off")


plt.tight_layout()
plt.show()
```

Travail demandé :

1. Modifier le programme pour afficher 20 images.
2. Organiser les images sur quatre lignes et cinq colonnes.
3. Observer les différences entre les écritures manuscrites.

Questions :

1. Les chiffres possèdent-ils toujours la même forme ?
2. Quelles difficultés un modèle pourrait-il rencontrer pour reconnaître certains chiffres ?
3. Pourquoi est-il nécessaire d'utiliser plusieurs exemples d'une même classe ?

Étape 6 — Analyse des pixels

Exécuter le programme suivant :

```
image = x_train[0]


print("Dimensions :", image.shape)


print("Valeur minimale :", image.min())
```

```
print("Valeur maximale :", image.max())


print("Matrice de pixels :")
print(image)
```

Travail demandé :

1. Observer les valeurs numériques représentant l'image.
2. Identifier les valeurs minimale et maximale.
3. Expliquer le rôle des pixels dans la représentation d'une image.
4. Déterminer le nombre total de pixels de cette image.

Étape 7 — Analyse de la distribution des classes

Pour connaître le nombre d'images appartenant à chaque classe, exécuter :

```
classes, effectifs = np.unique(
   y_train,
   return_counts=True
)


for chiffre, nombre in zip(classes, effectifs):
   print(f"Chiffre {chiffre} : {nombre} images")
```

Représenter graphiquement la distribution :

```
plt.figure(figsize=(8, 5))


plt.bar(classes, effectifs)


plt.xlabel("Chiffre")
plt.ylabel("Nombre d'images")
plt.title("Distribution des classes MNIST")
```

```
plt.xticks(range(10))


plt.show()
```

Travail demandé :

1. Identifier le chiffre le plus représenté.
2. Identifier le chiffre le moins représenté.
3. Comparer les effectifs des différentes classes.
4. Expliquer pourquoi la distribution des classes est importante dans un problème de classification.

5. Exercices d'application

Exercice 1 — Exploration d'une image

Écrire un programme Python permettant de :

1. Sélectionner l'image d'indice 50.
2. Afficher son étiquette.
3. Afficher ses dimensions.
4. Afficher ses valeurs minimale et maximale.
5. Visualiser l'image.

Compléter le programme suivant :

```
indice = 50


image = x_train[...]

etiquette = y_train[...]


print("Étiquette :", ...)

print("Dimensions :", ...)

print("Minimum :", ...)

print("Maximum :", ...)
```

```
plt.imshow(image, cmap="gray")

plt.show()
```

Exercice 2 — Recherche d'images appartenant à une classe

Écrire un programme permettant d'afficher cinq images représentant le chiffre 7.

Utiliser l'instruction suivante :

```
indices = np.where(y_train == 7)[0]
```

Travail demandé :

1. Identifier les indices des images représentant le chiffre 7.
2. Sélectionner cinq images.
3. Afficher les images dans une même figure.
4. Ajouter une étiquette à chaque image.

Question :

Les cinq chiffres présentent-ils exactement la même écriture ? Justifier votre réponse.

Exercice 3 — Comparaison de deux classes

Écrire un programme permettant de :

1. Calculer le nombre d'images représentant le chiffre 0.
2. Calculer le nombre d'images représentant le chiffre 9.
3. Calculer la différence entre les deux effectifs.
4. Déterminer lequel des deux chiffres est le plus représenté.

Code de départ :

```
nb_zero = np.sum(y_train == 0)


nb_neuf = np.sum(y_train == 9)


print("Nombre de 0 :", nb_zero)

print("Nombre de 9 :", nb_neuf)


# Compléter le programme
```

6. Synthèse et questions de compréhension

Répondre aux questions suivantes :

1. Quelle est la différence entre Machine Learning et Deep Learning ?
2. Qu'est-ce que l'apprentissage supervisé ?
3. Quelle est la différence entre classification et régression ?
4. Qu'est-ce qu'un pixel ?
5. Comment une image est-elle représentée dans un ordinateur ?
6. Pourquoi faut-il séparer les données d'entraînement et les données de test ?
7. Quel est l'intérêt d'analyser la distribution des classes avant l'entraînement d'un modèle ?

7. Travail à remettre

À la fin de la séance, chaque étudiant doit conserver son notebook contenant :

- Le chargement du jeu de données MNIST.
- L'exploration des dimensions.
- La visualisation des images.
- L'analyse des valeurs des pixels.
- La représentation graphique des classes.
- Les solutions des exercices proposés.